xGRAPH

October 3, 2025

1 Introduction

xGRAPH is a general purpose multidimensional batch graphics code. The results are graphed and output if required. The xgraph function call syntax is:

- **xgraph (data [,input])**

This takes simulation data and input cell arrays, then plots graphs. The data should have as many cells as there are input cells, for sequences.

If data = 'filename.mat', the specified file is read both for input and data. Here .mat indicates a Matlab file.

When the data input is a filename, parameters in the file can be replaced by new input parameters that are specified. Any stored input in the file is then overwritten when graphs are generated. This allows graphs of data to be modified retrospectively, if the simulation takes too long to be run again in a reasonable timeframe.

1.1 Parameter and data structures

This is a batch graphics function, intended to process quantities of graphics data, input as a cell array of multi-dimensional data. Theoretical and/or experimental data is passed to the graphics program, including the complete data cell array and a cell array of graphics parameters for plotting each graph.

To explain xGRAPH in full detail,

- Data to be graphed are recorded sequentially in a cell array, with data={d1,d2,...}.
- Graphics parameters including defaults are given in the input cell array.
- This describes a sequence of graph parameters, so that input={p1,p2,...}.
- For a one member sequence, a dataset and parameter structure can be used on its own.
- Each dataset and parameter structure describes a set of graphs.

The data input to xGRAPH can either come from a file, or from data generated directly with xSIM. The main graphics data is a nested cell array. It contains several numerical graphics arrays. Each defines one independent set of averaged data, the observed data averages, stored in a cell array indexed as $data\{s\}\{n\}(\ell, \mathbf{j}, c)$. To graph these also requires a corresponding cell array of structures of graphics parameters.

The output is unlimited, apart from memory limits. The program also generates error comparisons and chi-squared values if required. The data structure for input is as follows:

1. The input data is a cell array of datasets, which can be collapsed to a single dataset
2. The parameters are also a cell array of parameter structures, which can be collapsed to one structure
3. The dataset is a cell array of multidimensional graphs, each with arbitrary dimensionality.

4. The first or line index of each graph array allows multiple lines, with different line-styles
5. The last or check index of each graph array is optionally used for error and comparison fields.
6. Each graph array can generate multiple graphic plots, as defined by the parameters.

2 Parameter table

The complete cell array of the simulation data is passed to the xGRAPH program, along with graphics parameters for each observable, to create a graphics data structure. Graphics parameters have default values which are user-modifiable by editing the xgpreferences function. There is some duplication between the graphics and simulation parameters.

Some input parameters are global parameters for all graphs. However, most xGRAPH parameters are cell arrays indexed by graph index. These graphics parameters are individually set for each output that is plotted, using the cell index $\{n\}$ in a curly bracket. If present they replace the global parameters like labels.

If a graph index is omitted, and the parameter is not a nested array, the program will use the same value for all graphs. The axes, glabels, legends, lines, logs, and xfunctions of each graph are nested cell arrays, as there can be any number of lines and axis dimensions. In the case of the logs switch, the observable axis is treated as an extra dimension.

The plotted result can be an arbitrary function of the generated average data, by using the optional input gfunction. If this is omitted, the generated average data that is input is plotted.

Comparisons are plotted if present in the input data indexed by the last or check index c, with $c > errors$, where $errors = 3$ is the usual maximum value.

A table of the graphics parameters is given below.

Label	Default value	Description
chisqplot{n}	0	χ^2 plot options
cutoff	-1e100	Global probability cutoff for χ^2 estimates
cutoffs{n}	cutoff	Probability cutoff for n-th χ^2 estimate
diffplot{n}	0	Comparison difference plot options
errors	0	Index of last error field in data
esample{n}	1	Size and type of sampling error-bar
font{n}	18	Font size for graph labels
gfunction{n}	@(d,~) d{n}	Functions of graphics data
glabels{n}	{'t' ,'x' ,'y' ,'z'}	Graph-specific axis labels
graphs	$[1 : max]$	Vector of all the required graphs
gsqplot{n}	0	G^2 or maximum likelihood plot options
gtransforms{n}	ftransforms	Fourier switches for graph labels
headers{n}	''	Graph headers
images{n}	0	Number of movie images
imagetype{n}	0	Type of 3D image
klabels	{'\omega' ,'k_x' ,'k_y' ,'k_z'...}	Global transformed axis labels
legends{n}	{'label1',..}	Legends for multi-line graphs
limits{n}	{[lc1,uc1],[lc2,uc2]}	Plotted axis limits, first lower then upper
linestyle{n}	{'-',..}	Line styles for multiline 2D graphs
linewidth{n}	0.5	Line width for 2D graphs (in points)
logs{n}	{0,..}	Axis logarithmic switch: 0 linear, 1 log
minbar{n}	0.01	Minimum relative error-bar
mincount	10	Global counts for chi-square cutoffs
name	''	Global graph header
numberaxis	0	Switch for numbering the graphics axes
octave	0	0 for Matlab, 1 for octave environment
olabels{n}	'a_1'	Observable labels
parametric{n}	0	Indicates a parametric plot
pdimension{n}	3	Maximum plot dimensions
savefig	0	Switch, set to 1 to save figure files
savegraph	0	Set '.pdf', '.pdf', '.png', '.jpg' or '.tif' to save
scale{n}	1	Scaling: Counts/ probability density
transverse{n}	0	Number of transverse plots
verbose	0	0 for brief, 1 for informative, 2 for full output
xfunctions{n}	{@(t,~) t,@(x,~) x,..}	Axis transformations
xlabels	{'t' ,'x' ,'y' ,'z'...}	Global axis labels
xk{n}	$\{[t_1, \ldots t_n], [x_1, \ldots x_n]...\}$	Axis numerical values

- Up to 6 types of input data can occur, including errors and comparisons, indexed by the last index. The original mean data always has c =1. If there are no errors or comparisons, one graph is plotted for each dimensional reduction.

- The input data has up to two error bars (I and II), and optional comparisons also with up to two error bars.

- Type I errors labeled $c = 2$ have standard vertical error bars. Type II errors labeled $c = 3$, which are usually standard deviation errors from sampling, have two solid lines.

- If esample = -1, both error bars are combined and the RMS errors are plotted as a single error bar.

- If $diffplot > 0$, differences are plotted as unnormalized ($diffplot = 1$), or normalized ($diffplot = 2$) by the total RMS errors. If $diffplot = 3$, raw comparison data is plotted.

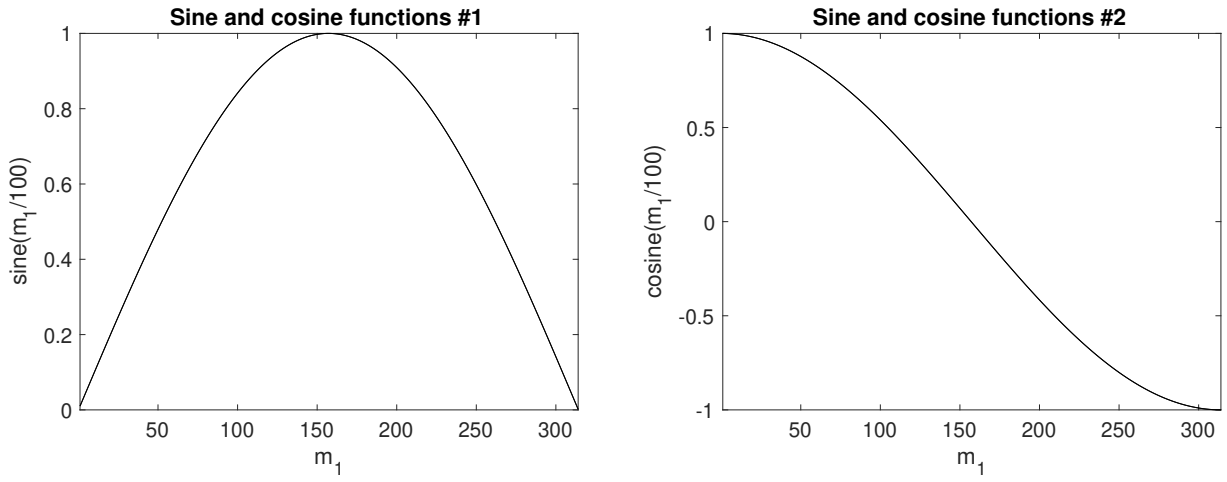


Figure 1: *Example: xgraph output of two plots.*

- When differences are plotted, the total comparison errors are treated as type I error bars, while total simulation errors are treated as type II errors with parallel lines in the graphs, in order to distinguish them.

Saved graphics files appear in the Matlab current directory. A detailed description of each parameter is listed in Sec (3).

2.1 Example

A simple example of data and input parameters, but without errors or comparisons is as follows

```
p.name = 'Sine and cosine functions';
p.olabels = {'sine(m_1\pi/100)','cosine(m_1\pi/100)'};
data = {sin([1:100*pi]/100),cos([1:100*pi]/100)};
xgraph(data,p);
```

Note that in this case the default setting of p.errors=0 is used, with no check index used in the data arrays, because these are simple graphs without error-bars or comparisons.

2.2 xGRAPH data arrays

The data input to xGRAPH can come from a file, or from data generated directly from any compatible program.

The data is stored in a cell array *data* with structure:

$$data\{s\}\{n\}(\ell, \mathbf{j}, c)$$

Each member of the outer cell array data{s} defines a number of related sets of graphical data, all described by common parameters input{s}. Comparisons and errors are plotted if there are errors and comparison data in the input, indexed by c. This generates comparison plots, as well as error totals and χ- squared error estimate when there are statistical variances available.

An individual member of data{s}{n} is a multidimensional array, called a graph. For each graph, multiple different plots with different dimensionality can be obtained from the dataset data{s}{n}, either through projections and slices or by generating additional data defined with graphics functions. Either or both alternatives are available.

Note that:

- If a sequence has one member, the outer cell array can be omitted.

- In this simplified case, if there is only one graph array, the inner cell array can be omitted.

The graphics data for a single dataset is held in a multidimensional real array, where:

- ℓ is the index for lines in the graph. Even for one line, the first dimension is retained.
- $\mathbf{j} = j_1, \ldots j_d$ is the array index in each dimension, where $d \geq 1$.
- Averages in momentum space have the momentum origin as the central index.
- If integrals or spatial averages are used, the corresponding dimension has one index $j_d = 1$.
- With probabilities, extra dimensions are added to $\mathbf{j}$ to store the bin indices.
- c indexes error-checks and comparisons. If not present, omit p.errors and the last dimension.
- If $c > p.errors$, the extra fields are comparison inputs, where $p.errors$ is the largest data index.

When the optional comparison fields are used, an input parameter $errors$ is required to indicate the maximum error index, to distinguish data from comparisons. Parameter structures from xSIM have $errors = 3$ set to allow for both sampling errors and discretization errors. If this is omitted, the default is $errors = 0$, which implies that there is no error or comparison data

If $errors > 0$, the last index can have larger values with $c > errors$, for comparisons. The special case of $errors = 1$ is used if the data has no error bars, but there are comparisons in the data. Larger indices are used to index the comparison data, which can also have two types of errors. The largest usable last index is $errors + 3$.

It is possible to directly plot the raw data using xGRAPH. One can even combine the raw data with a graphics parameter input. But since the raw data has no error estimates - it is raw data - one must set $p.errors = 0$, since the xsim output parameters have a normal setting of $p.errors = 3$. This will give a single trajectory.

However, the raw data from a simulation typically includes many trajectories if $ensembles(1) > 0$. One must select particular trajectory datasets from the raw cell array, to plot just one.

2.3 Input parameters and defaults

A sequence of graph parameters is obtained from inputs in a cell array, as input = {in1, in2, ...}. The input parameters of each simulation in the sequence are specified in a Matlab structure. The inputs are numbers, vectors, strings, functions and cell arrays. All metadata has preferred values, so only changes from the preferences need to be input. The resulting data is stored internally as a sequence of structures in a cell array, to describe the simulation sequence.

The graphics parameters are also stored in the cell array input as a sequence of structures p. This only need to be input when the graphs are generated and can be changed at a later time to alter the graphics output. A sequence of simulations is graphed from input specifications.

If there is one simulation, just one structure can be input, without the sequence braces. The standard way to input each parameter value is:

$$p.label = parameter$$

The standard way to input a function handle is:

$$p.label = @function$$

The inputs are scalar or vector parameters or function handles. Quantities relating to graphed averages are cell arrays, indexed by the graph number. The available inputs, with their default values in brackets, are given below.

Simulation metadata, including default values that were used in a particular simulation, can be included in the input data files. This is done in both the .mat and the .h5 output files generated by xSIM, so the entire graphics input can be reconstructed or changed.

Parameters can be numbers, vectors, strings or cell arrays. Conventions that are used are that:

- All input parameters have default values
- Vector inputs of numbers are enclosed in square brackets, [...].
- Cell arrays of strings, functions or vectors are enclosed in curly brackets.
- Vector or cell array inputs with only one member don't require brackets.
- Incomplete parameter inputs are completed with the last used default value.
- Function definitions can be handles pointing elsewhere, or defined inline.

If any inputs are omitted, there are default values which are set by the internal function xgpreferences. The defaults can be changed by editing xgpreferences.

In the following descriptions, graphs is the total number of graphed variables of all types. The space coordinate, image, image-type and transverse data can be omitted if there is no spatial lattice, that is, if the dimension variable is set to one.

For uniformity, the graphics parameters that reference an individual data object are cell arrays. These are indexed over the graph number using braces {}. If a different type of input is used, like a scalar or matrix, xGRAPH will attempt to convert the type to a cell array.

Axis labels are cell arrays, indexed over dimension. The graph number used to index these cell arrays refers to the data object. In each case there can be multiple generated plots, depending on the graphics input.

2.4 Cascaded plots

The xGRAPH function generates a default range of graphs, but this can be modified to suit the user. In the simplest case of one dimension, one graph dataset will generate a single plot. For higher dimensions, a cascade of plots is generated to allow visualization, starting from 3D movies, then 3D static plots and finally 2D slices. These can also be user modified.

Note that for all probabilities, the plot dimension is increased by the bin range dimensionality.

Plot dimensions The pdimension input sets the maximum plotted dimensions. For example, $pdimension\{1\} = 1$ means that only plots vs r_1 are output for the first function plotted. Default values are used for the non-plotted dimensions, unless there are axes specified, as indicated below.

The graphs cascade down from higher to lower dimensions, generating different types of graphs. Each type of graph is generated once for each function index.

Plot axes The graphics axes that are used for plotting and the points plotted are defined using the optional axes input parameters, where $axes\{n\}$ indicates the n-th specified graph or set of generated graph data.

If there are no axes inputs, or the axes inputs are zero - for example, $axes\{1\} = \{0, 0, 0\}$ - only the lowest dimensions are plotted, up to 3. If either the data or axes inputs project one point in a given dimension, - for example, $axes\{1\} = \{0, 31, -1, 0\}$, this dimension is suppressed in the plots, which reduces the effective dimension of the data - in this case to two dimensions.

Examples:

- $axes\{1\} = \{0\}$ - For function 1, plot all the first dimensional points; higher dimensions get defaults.
- $axes\{2\} = \{-2, 0\}$ - For function 2, plot the maximum value of r_1 (the default) and all higher-dimensional x-points.
- $axes\{3\} = \{1 : 4 : 51, 32, 64\}$ - For function 3, plot every 4-th x_1 point at x_2 point 32, x_3 point 64
- $axes\{4\} = \{0, 2 : 4 : 48, 0\}$ - For function 4, plot every x_1 point , every 4-th x_2 point, and all x_3-points.

Points labelled -1 indicates a default 'typical' point, which is the midpoint. If one uses -2, this is the last point.

Lower dimensions are replaced by corresponding higher dimensions if there are dimensions or axes that are suppressed. Slices can be taken at any desired point, not just the midpoint. The notation of $axes\{1\} = \{6 : 3 : 81\}$, is used to modify the starting, interval, and finishing points for complete control on the plot points.

The graphics results depend on the resulting **effective** dimension, which is equal to the actual input data dimension unless there is an axes suppression, described above. Since the plot has to include a data axis, the plot itself will usually have an extra data axis.

One can plot only three axes directly using standard graphics tools. The strategy to deal with the higher effective dimensionality is as follows. For simplicity, "time" is used to label the first effective dimension, although in fact any first dimension is possible:

dimensions = 1 For one lattice dimension, a 2D plot of observable vs t is plotted, with data at each lattice point in time. Exact results, error bars and sampling error bounds are included if available.

dimensions = 2 For two lattice dimensions, a 3D image of observable vs x,t is plotted. A movie of distinct 2D graphic plots is also possible. Otherwise, a slice through $x = 0$ is used tp reduce the lattice dimension to 1.

dimensions = 3 For three lattice dimensions, if $images > 1$, a movie of distinct 3D graphic images of observables are plotted as *images* slices versus the first plot dimension. Otherwise, a slice through the chosen point, is used at the highest dimension to reduce the lattice dimension to 2.

dimensions = 4,5.. For higher lattice dimensions, a slice through a chosen point, or the default midpoint is used to reduce the lattice dimension to 3.

As explained above, in addition to graphs versus x_1 the **xGRAPH** function can generate images (3D) and transverse (2D) plots at specified points, up to a maximum given by the number of points specified. The number of these can be individually specified for each graph number. The images available are specified as imagetype= $1, \ldots 4$, giving:

1. 3D perspective plots (Matlab surf - the default)
2. 2D filled color plots (Matlab contourf)
3. contour plots (Matlab contour)
4. pseudo-color plots (Matlab pcolor)

Error bars, sampling errors and multiple lines for comparisons are only graphed for 2D plots. Error-bars are not plotted when they are below a user-specified size, with a default of 1% of the maximum range, to improve graphics quality. Higher dimensional graphs do not output error-bar data, but they are still recorded in the data files.

2.5 Chi-squared and G-squared plots

Probability data can be input and plotted like any other data. It is typically generated from simulation programs using the *binranges* data for binning. It is plotted like any other graph, with any dimension, except that the total dimension is extended by the number of variables or lines in the observe function.

Chi-squared data is output interactively if $p.verbose > 0$, provided there is comparison data in the input.

Chi-squared plots In addition the program can make a χ^2 plot, which is a plot of the χ^2 comparison with a comparison probability density against space and/or time, assuming Gaussian statistics. This allows a test of the simulated data against a known target probability distribution, provided that the following input data conditions are satisfied:

- The input data dimension exceeds the p.dimensions parameter,
- The switch p.chisqplot is set to 1or 2, and
- The input data includes comparison function data.

Here, for one point in space and time, with m bins, N_j counts per bin and E_j expected counts:

$$\chi^2 = \sum_{j=1}^{m} \frac{(N_j - E_j)^2}{E_j}. \tag{1}$$

The number k is the number of valid counts, with $N_j, E_j > mincount$. This is partly determined from the requirement that the probability count data per bin is greater than the *p.mincount* parameter. The default is set to give a number of samples > 10. The program prints a summary that sums over of all the χ^2 data.

The χ^2 plots, depending on *p.chisqplot* are:

1. a plot of χ^2 and k, where k is the number of valid data points,
2. a plot of $z = \left[\left(\chi^2/k\right)^{1/3} + \frac{2}{9k} - 1\right] / \sqrt{\frac{2}{9k}}$, the Wilson-Hilferty z-value, which has a unit variance.

The $p.scale\{n\}$ parameter gives the number of counts per bin at unit probability density. This is needed to set the scale of the χ^2 results, ie, $N_j = scale\{n\} \times p_j$, where p_j is the probability density that is compared and plotted in the simulation data. Note that a uniform bin size is assumed here, to give a uniform scaling.

Comparisons with variances It can be useful to compare two probability distributions with different variances. For one point in space and time, with m bins, p_j probability density and e_j expected probability density,

$$\chi^2 = \sum_{j=1}^{m} \frac{(p_j - e_j)^2}{\sigma_j^2 + \sigma_{e,j}^2}. \tag{2}$$

In this case, σ_j^2 and $\sigma_{e,j}^2$ are the sampling errors in the simulation data and comparison data, so that built-in error fields in the data are used to work out the χ^2 results. This option is chosen if $p.scale\{n\} = 0$, and the cutoff for the data is then specified so that $p_j, e_j > p.cutoffs\{n\}$. The default value is the global cutoff, $p.cutoff$, which has a default of 10^{-12}.

This output only has a χ^2 distribution with $\chi^2 \approx m$ if all the points are independent. The measured χ^2/m includes all the space-time points above the cutoff value. This can be less than unity when comparing an expected exact result with a computed SDE solution where data is often correlated. At the other extreme, if the cutoff is too low, the data may not be reliable, and one can obtain too large a value.

Maximum likelihood It is also possible to plot the G^2 or maximum likelihood plot of the data, which is an alternative means to compare distributions, where

$$G^2 = 2 \sum_{j=1}^{m} N_j \ln\left(N_j / E_j\right). \tag{3}$$

The expected values E_j are automatically scaled so that $\sum N_j = \sum E_j$,with the same minimum count cutoff that is used for the χ^2 data. The result is similar to the χ^2 results. It is obtained if p.gsqplot is set to 1 or 2 and requires for the input that $p.scale\{n\} > 0$. It is sometimes regarded as a preferred method for comparisons.

2.6 Parametric plots

Any input dataset can be converted to a parametric plot, where a second data input is plotted along the horizontal axis instead of the time coordinate. It is also possible to substitute a second data input for the x-axis data if a parametric plot in space is required instead. This allows visualization of how one type of data changes as a function of a second type of data input.

The two datasets that are plotted must have the same number of lines, that is, the first index range should be the same, in order that multiple lines can be compared. This is achieved where required using the p.scatters input in the simulation code. The details of the parametric plot are specified using the input:

$$p.parametric\{n\} = [n1, p2] \tag{4}$$

Here n is the graph number which is plotted, and must correspond to an input dataset. The number $n1$ is the graph number of the observable that is plotted on the horizontal axis, ignoring functional transformations. The second number is the axis number where the parametric value is substituted, which can be the time (axis 1) or the x-coordinate (axis 2), if present.

In all cases the vertical axis is used to plot the original data. The specified horizontal axis is used for the parametric variable. Only vertical error-bars are available.

3 xGRAPH Parameter reference

3.1 chisqplot{n}

Default: 0

Plots χ^2 and k values for probabilities, to show how these evolve in time and/or space. Use p.chisqplot{n} = 1 for a direct plot of χ^2 and k ; p.chisqplot{n} = 2 gives a Wilson-Hilferty transformed z-value.

Example: p.chisqplot{3} = 2

3.2 cutoff

Default: $-1.e^{-20}$

Defines a cutoff so that low probability data can be ignored when calculating χ^2 and k values, and graphing probabilities. Also see mincount.

Example: p.cutoff =$1.e^{-7}$

3.3 cutoffs{n}

Default: $p.cutoff$

Graph-dependent cutoff so low probability data is ignored when calculating χ^2 and k values and graphing probabilities.

Example: p.cutoffs{1} =$1.e^{-7}$

3.4 diffplot{n}

Default: 0

Differences are plotted as a comparison dashed line on $2D$ plots as a default. Otherwise, a separate difference plot is obtained which is unnormalized (diffplot = 1), or normalized (diffplot = 2) by the total RMS errors. If diffplot = 3, the comparison data is plotted directly as an additional graph.

Example: p.diffplot{3} = 2

3.5 errors

Default: 0

Indicates if the last index in the graphics input data arrays is used for error-bars and/or comparisons. Should be set to zero if there is no error or comparison data. If non-zero, this will give the highest last index used for errors. The standard xsim output sets $p.errors = 3$ automatically. As a special case, $p.errors = 1$ is used to indicate that there is comparison data but no error data.

If $p.errors > 0$, the data indexed up to p.errors gives the data, then a maximum of two types of error bars. Up to three further index values, up to $p.errors + 3$, are available to index all comparison data and its error fields. The maximum last index value used is 6.

Example: p.errors = 2

3.6 esample{n}

Default: 1

This sets the type and size of sampling errors that are plotted. If esample = 0, no sampling error lines are plotted, just the mean. If $esample = -n$, $\pm n\sigma$ sampling errors are included in the error-bars. If $esample = n$, separate upper and lower $\pm n\sigma$ sampling error lines are plotted. In both cases, the magnitude of esample sets the number of standard deviations used.

Example: p.esample{3} = -1

3.7 font{n}

Default: 18

This sets the default font sizes for the graph labels, indexed by graph. This can be changed per graph.

Example: p.font{4}=18

3.8 gfunction{n} (d,p)

This is a cell array of graphics function handles. Use when a graph is needed that is a functional transformation of the observed averages. The default value generates the n-th graph data array directly from the n-th input data. The input is the data cell array for all the graphs in the current sequence number with their graph parameters x, and the output is the n-th data array that is plotted.

An arbitrary number of functions of these observables can be plotted, including vector observables. The input to graphics functions is the observed data averages or functions of averages in a given sequence, each stored in a cell array $d\{n\}(\ell, \mathbf{j}, c)$. If there are more graphics functions than input data cells, this generate additional data for plotting.

3.9 glabels{n}

Default: xlabels or klabels

Graph-dependent labels for the independent variable labels. This is a nested cell array with first dimension of graphs and second dimension of dimensions. This is used to replace the global values of xlabels or klabels if the axis labels change from graph to graph, for example, if the coordinates have a functional transform. These can be set for an individual coordinate on one graph if needed.

Example: p.glabels{4}{2} = 'x^2'

3.10 graphs

Default: observables to plot

This gives the observables to plot. The default is a vector of indices from one to the length of the cell array of observe functions. Normally not initialized, as the default is used. Mostly used to reduce graphical output on a long file.

Example: p.graphs = 10

3.11 gsqplot{n}

Default: 0

Plots maximum likelihood or G^2 and k values for probabilities, to show how these evolve in time and/or space. Use p.gsqplot{n} = 1 for a plot of χ^2 and k ; p.gsqplot{n} = 2 gives a Wilson-Hilferty transformed z-value.

Example: p.gsqplot{3} = 2

3.12 gtransforms{n}

Default: [0,0,...]

This switch specifies the Fourier transformed graphs and axes for graphics labeling. Automatically equal to ftransforms if from an earlier xSIM input, but can be changed. If altered for a given graph, all the axis Fourier switches should be reset. This is ignored if there is no dimensions setting to indicate space dimensions.

Example: p.gtransforms{1} = [0,0,1]

3.13 headers{n}

Default: ”

This is a string variable giving the graph headers for each type of function plotted. The default value is an empty string. Otherwise, the header string that is input is used. Either is combined with the simulation name and a graph number to identify the graph. This is used to include simulation headers to identify graphs in simulation outputs. Graph headers may not be needed in a final published result. For this, either edit the graph, or use a space to make plot headers blank: p.headers{n} = ’ ’, or p.name = ’ ’ .

Example: p.headers{n} = ’my_graph_header’

3.14 images{n}

Default: 0

This is the number of 3D, transverse o-x-y images plotted as discrete time slices. Only valid if the input data dimension is greater than 2. If present, the coordinates not plotted are set to their central value when plotting the transverse images. This input should have a value from zero up to a maximum value of the number of plotted points. It has a vector length equal to graphs.

Example: p.images{4} = 5

3.15 imagetype{n}

Default: 1

This is the type of transverse o-x-y movie images plotted. It has a vector length equal to graphs.

- imagetype = 1 gives a perspective surface plot
- imagetype = 2, gives a 2D plot with colors
- imagetype = 3 gives a contour plot with 10 equally spaced contours
- imagetype = 4 gives a pseudo-color map

Example: p.imagetype{n} = 1, 2, 3, 4

3.16 klabels

Default: {'\omega', 'k_x', 'k_y', 'k_z'}" or "{'k_1', 'k_2', 'k_3', 'k_4',...}

Labels for the graph axis Fourier transform labels, vector length of dimensions. The numerical labeling default is used when the "p.numberaxis" option is set. Note, these are typeset in Latex mathematics mode! When changing from the default values, all the required new labels must be set.

Example: p.klabels= {'\Omega', 'K_x', 'K_y',}

3.17 legends{n}

Default: {","}

Graph-dependent legends, specified as a nested cell array of strings for each line.

Example: p.legends{n} = {labels(1), ..., labels(lines)}

3.18 limits{n}

Default: {0,0,0,0; ...}

Graph-dependent limits specified as a cell array with dimension graphs. Each entry is a cell array of graph limits indexed by the dimension, starting from $d = 1$ for the time dimension. The limits are vectors, indexed as 1,2 for the lower and upper plot limits. This is useful if the limits required change from graph to graph. If an automatic limit is required for either the upper or lower limit, it is set to inf.

An invalid, scalar or empty limit vector, like [0,0] or 0 or [] is ignored, and an automatic graph limit is used.

Example: p.limits{n} = {[t1,t2],[x1,x2],[y1,y2] ...,}

3.19 linestyle{n}

Default: {'-k','--k',':k','-.k','-ok','--ok',':ok','-.ok','-+k','--+k'}

Line types for each line in every two-dimensional graph plotted. If a given line on a two-dimensional line is to be removed completely, set the relevant line-style to zero. For example, to remove the first line from graph 3, set p.linestyle{3} ={0}. This is useful when generating and changing graphics output from a saved data file. The linestyle uses Matlab terminology. It allows setting the line pattern, marker symbols and color for every line. The default lines are black ('k'), but any other color can be used instead.

The specifiers must be chosen from the list below, eg, '-ok', although the marker can be omitted if not required.

- Line patterns: '-' (solid), '–' (dashed), ':' (dotted) ,'-.' (dash-dot)
- Marker symbols: '+','o','*','.','x','s','d','^','v','>','<','p'
- Colors: 'r','g','b','c','m','y','k','w'

Example: p.linestyle{4} = {'-k','--ok',':g','-.b',}

3.20 linewidth{n}

Default: 0.5

Line width for plotted lines in two-dimensional graphs. For example, to make the lines wider in graph 3, set p.linewidth{3} =1. This is useful for changing graphics output appearance if the default lines are too thin.

Example: p.linewidth{n} = 1

3.21 logs{n}

Default: {0,..}

Switch for logarithmic transforms of axes. The last axis is the variable or function plotted.

Example: p.logs{n} = {1,1};

3.22 minbar{n}

Default: {0.01, ...}

This is the minimum relative error-bar that is plotted. Set to a large value to suppress unwanted error-bars, although its best not to ignore the error-bar information! This can be changed per graph.

Example: p.minbar{n} = 0

3.23 mincount{n}

Default: {10, ...}

This is the global cutoff used obtain a minimum probability for a χ^2 calculation or a G^2 calculation.

Example: p.mincount{n} = 0

3.24 name

Default: ''

Name used to label simulation graphs, usually corresponding to the equation or problem solved. This can be removed from individual graphs by using headers{n} equal to a single blank space. The default is a null string. To remove all headers globally, set name equal to a single blank space: name = ' '.

Example: p.name = 'Wiener process simulation'

3.25 numberaxis

Default: 0

Switch for numbering graphics axes numerically. Otherwise the labels $t, x, , y, z$ are used, unless the total dimensions are too large.

Example: p.numberaxis = 1

3.26 octave

Default: 0

Switch for Octave program syntax, which can differ from Matlab standards. Octave is an open source, free alternative to Matlab.

Example: p.octave = 1

3.27 olabels{n}

Default: 'a'

Cell array of labels for the graph axis observables and functions. These are text labels that are used on the graph axes. The default value is 'a_1' if the default observable is used, otherwise it is blank. This is overwritten by any subsequent label input when the graphics program is run:

Example: p.olabels{4} = 'v'

3.28 parametric{n}

Default: [0,0]

Cell array that defines parametric plots, for each graph number. The first number is the graph number of the alternative observable plotted on the horizontal axis. The second number is the axis number where the parametric value is substituted, which can be the time (axis 1) or the x-coordinate (axis 2), if present.

If both are zero, the plot against an independent space-time coordinate is calculated as usual. If nonzero, a parametric plot is made for two-dimensional plots. In all cases the vertical axis is used to plot the original data. The specified horizontal axis is used for the parametric variable. Only vertical error-bars are available. Can be usefully combined with scatters{n} to plot individual trajectories, but the number of scatters should be the same in each of the two graphs that are parametrically plotted against each other.

Example: p.parametric{n} = [p1,p2] >= 0

3.29 pdimension{n}

Default: 3

This is the maximum plotted space-time dimension for each plotted quantity. The purpose is eliminate unwanted graphs. For example, it is useful to reduce the maximum dimension when averaging in space. Higher dimensional graphs are not needed, as the data is duplicated. Averaging can be useful for checking conservation laws, or for averaging over homogeneous data to reduce sampling errors. All graphs are suppressed if it is set to zero. Any three dimensions can be chosen to be plotted, using the axes parameter to suppress the unwanted data points in other dimensions.

Example: p.pdimension{4} = 2

3.30 savefig

Default: 0

If set to a string ending in .fig , all graphs are saved to the current folder as .fig files, numbered consecutively as Fig1.fig, Fig2.fig.. . Use *close all* to remove existing graphs, before running xgraph. The total number of files saved includes subfigures, and may be greater than the number of '*observe*' functions. A descriptive string before the ending can be used to identify the dataset.

Example: p.savefig = 'gx.fig' saves Matlab files with names Fig1gx.fig, Fig2gx.fig...).

3.31 savegraph

Default: 0

If set to a string ending in '.jpg', '.pdf', '.png', or '.tif' all plots are saved to the current folder as .jpg (etc) files, numbered as Fig1.jpg, and so on. Use the *close all* command first to remove unwanted displayed graphs, before running xgraph with this option, since all displayed graphs are stored. A descriptive string before the ending can be used to identify the dataset.

Example: p.savegraph = 'gx.png' saves png files with names Fig1gx.png, Fig2gx.png...).

3.32 scale{n}

Default: 1

Counts/probability density: used for χ^2 and G^2 calculations to scale counts for cutoffs in probabilities. It is defined as: $scale = (bin\ area) \times (trajectories)$. Here trajectories is the total trajectories used for binning, i.e., prod(ensembles).

Example: scale{1} = 0.01 .

3.33 transverse{n}

Default: 0

This is the number of 2D transverse images plotted as discrete time slices. Only valid if dimensions is greater than 2. If present, the y, z-coordinates are set to their central values when plotting transverse images. Each element can be from 0 up to the number of plotted time-points. The cell array has a vector length equal to graphs.

Example: p.transverse{n}= 6

3.34 verbose

Default: 0

Print flag for output information while running xGRAPH. Print options are:

- Minimal if verbose = -1: Prints just the start-up time and hard error messages
- Brief if verbose = 0: Additionally prints the final, total chi-squared errors where present
- Informative if verbose = 1: Also prints the graph progress indicators
- Full if verbose = 2: Prints everything including the internal parameter structure data.

In summary, if verbose = 0, most output is suppressed except the final data, verbose = 1 displays a progress report, and verbose = 2 additionally generates a readable summary of the graphics parameter input.

Example: p.verbose = 0

3.35 xlabels

Default: {'t', 'x', 'y', 'z'} or {'x_1', 'x_2', 'x_3', 'x_4',...}

Global labels for the independent variable labels, vector length equal to dimensions. The numerical labeling default is used when the numberaxis option is true. These are typeset in Latex mathematics mode. When changing from the default values, all the required new labels must be set.

Example: p.xlabels = {'tau'}

4 xGRAPH structure

The graphics function, *xgraph*, plots the simulation data. The general structure is:

$$\begin{aligned}\textbf{xgraph} &\rightarrow \textbf{xgpreferences}\ (\textit{checks inputs})\\ &\rightarrow \textbf{xmultigraph} \leftrightarrow \textbf{xreduce} \leftrightarrow \textbf{xcompress}\ (\textit{structures data arrays})\\ &\rightarrow \textbf{ximages} \rightarrow \textbf{xtransverse} \rightarrow \textbf{xplot3} \rightarrow \textbf{xplot2}\ (\textit{graphs all data})\end{aligned}$$

Most graphics functions simply work, but two important functions are listed below for reference.

4.1 xgraph(data,input)

The xgraph function graphs multidimensional data files.

- Input: graphics data cells data, input parameter cells input.
- Output: graphs, displayed and/or stored as eps or fig files.
- If no numeric data present, reads data from a file named data.
- If data is present but without any input parameters it plots using default parameters.
- First data dimension is the line index, last dimension are the error-bars and comparisons
- Needs: xread, xmakecell, xgpreferences, xmultiplot

4.2 xgpreferences (input,oldinput)

The xgpreferences function sets default values for graphics inputs.

- Input: input cell array and optionally previous inputs from a datafile, oldinput.
- Note that each cell array is a sequence of graphics parameter structures
- Output: the updated plus default graphics parameters
- Called by: xgraph
- Needs: xprefer, xcprefer